The `posim` Graphical Scene Window

Research, Design & Complete Guide (`scene_info`)

physical_object_simulator

July 22, 2026

This is the typeset twin of `scene_info.md` — companion to `grammar.pdf` and `physical_object_simulator.pdf`.

This document explains, for a reader with **zero prior knowledge**:

1. what the graphical scene window is and how to drive it,
2. the **research** performed on seven established physics simulators before designing it,
3. a **comparison chart and concept tree** of that "scene science",
4. which ideas we recommend (and chose) to **port to pure Rust**, and
5. the full technical documentation of the implementation, its asynchronous notebook link, and how every claim was **verified**.

Contents

1 What is the scene window?

`posim` is a physics simulator you drive by typing commands (`NEW SPHERE { ... }`, `STEP 0.1`, ...). Numbers are exact but hard to *picture*. The **scene window** is a live 3-D view of every object in your simulation that opens **in your web browser, outside the notebook**, the moment you type:

```
scene create
```

You get back something like:

```
scene window created: http://127.0.0.1:41234/
(opened in your browser; if no window appeared, open that address yourself)
showing 4 entities; SCENE START begins the evolution - HELP lists all scene commands
```

The window shows **all simulator entities** (points, spheres, cuboids, tori, disks, cylinders, dumbbells), a ground grid, the world axes, motion trails, and object labels (an object registered with `NEW ... AS <name>` is labeled by that name). If the notebook created a rigid bounding box (`BOX <size>`), the window draws its interior as a dashed light-blue wireframe — the six infinitely-massive wall slabs that implement it are represented by that wireframe, never drawn as six giant cuboids. The window keeps itself up to date over a private network link to the simulator — you never copy data by hand. It can also *talk back*: errors and user actions inside the window flow to the notebook asynchronously (§7).

No extra software is needed: the whole server is built into `posim` itself using only the Rust standard library — zero external dependencies, zero `unsafe` code.

1.1 The toolbar (top of the window)

control	what it does
Start	begin time-stepped evolution, forward in time
Pause	freeze the evolution (resume with Start)
Stop	halt evolution *and clear the recorded history*
Reverse	play **backward** through the recorded history
Reset	re-initialize the playback: **every value and the time return to their initial values**; Start then runs the simulation again from the beginning
step back / step	single step backward / forward
dt `____` Set	change the playback time step
zoom + / zoom –	zoom in / out
View	reset the camera to its home position
Grid / Trails / Labels	toggle the ground grid, motion trails, object labels
Contacts	toggle contact-normal arrows drawn at collision points
?	show the controls cheat-sheet

1.2 The "conserved quantities" readout (top-left)

A permanent labeled overlay (just below the toolbar) shows the three conserved quantities of the playback copy, updated live at every frame: **E** (total energy), **P** (total linear momentum) and **L** (total angular momentum about the origin). P and L are printed as their three components plus the magnitude `|.|`. Watching E, P and L sit still through a collision *is* the physics lesson — the values come straight from the frame protocol's `energy`, `p` and `l` fields (§5.1), computed on the playback copy every tick.

1.3 The status bar (bottom of the window)

A live one-line dashboard: **connection dot** (green = connected to posim), playback **mode** (`stopped` / `running` / `paused` / `reversing`), simulated time **t**, time step **dt**, total energy **E**, **bodies** count, **hidden** count, **contacts** count (of the latest playback step), **history** (frames available for Reverse), the **camera** yaw/pitch/distance, and the rendering **fps**.

1.4 Controls (mouse & keyboard) — all verified in a real browser

gesture	effect
left / right arrow keys	**translate** the view left / right
up / down arrow keys	translate the view up / down
left-click + drag	**rotate** (orbit) around the scene
mouse wheel	**zoom** in / out
`+` / `-` keys	zoom in / out (documented keyboard shortcut)
Shift+drag or right-drag	translate (pan) with the mouse
Space	start / pause playback
`R`	reset the view
`G` / `T` / `L`	toggle grid / trails / labels
`C`	toggle contact-normal arrows
`H` or `?`	toggle the help overlay

2 The research: seven simulators investigated

Before writing a line of code we sent research agents to study how seven established, widely-used simulators present and control a graphical scene. A summary of each, then the comparison chart.

2.1 PyChrono / Project Chrono (C++, Python bindings)

Multi-physics engine used in engineering and robotics. Rendering is **in-process** through an abstract *visual system* layer with pluggable backends — Irrlicht (`ChVisualSystemIrrlicht`), VulkanSceneGraph (`ChVisualSystemVSG`), raw OpenGL, plus offline Blender/POVRay export. Typical loop: `AttachSystem(sys)`, `Initialize()`, then `Run()/BeginScene()/Render()/EndScene()` around `ChSystem::DoStepDynamics(dt)` — *you* own the loop and the time step; pause/step are gates you place around `DoStepDynamics`. Scene decoration: `AddCamera(pos, target)`, `AddTypicalLights()`, `AddSkyBox()`, `EnableShadows()`. Viewer: left-drag rotate, right-drag pan, wheel zoom, `i` info panel, Space pause.

2.2 PyBullet (C++ engine, Python module)

The gold standard in robotics labs. **Client–server by design**: a client sends *command structs*, the physics server returns *status structs* — identical code whether the transport is same-process (`connect(DIRECT)`/`connect(GUI)`), shared memory, UDP, or TCP. Key calls: `stepSimulation()`, `setTimeStep(dt)`, `setRealTimeSimulation(0/1)`, `resetDebugVisualizerCamera(distance, yaw, pitch, target)`, `configureDebugVisualizer(flag, on)`, `addUserDebugLine/Text/Parameter`, `getKeyboardEvents()` / `getMouseEvents()`. The GUI has dockable parameter panels and preview tiles. No built-in reverse.

2.3 SOFA (C++, Qt/OpenGL GUI)

Interactive multi-physics (deformables, biomechanics). `runSofa` is an **in-process Qt app**; the default viewport is libQGLViewer (`QtGLViewer`). Scenes are declarative trees (`.scn` XML or Python). GUI: **Animate**, **Step**, **Reset Scene** buttons and an editable **dt** field; FPS + elapsed time bottom-left; Save/Reset View. Mouse: left rotate, right pan, wheel zoom, Shift+click pokes the simulation. Shortcuts: `S` screenshot, `T` ortho/perspective, `V` video. No reverse.

2.4 Rapier (Dimforge, pure Rust)

Modern 2-D/3-D physics engines in Rust. The core is **fully rendering-agnostic**: `PhysicsPipeline::step()` advances `RigidBodySet`/`ColliderSet`; you read poses back through handles. The testbed (Bevy + egui) contributes the clearest control vocabulary: `RunMode {Running, Stop, Step}`, display bitflags (`TestbedStateFlags`: `WIREFRAME`, `AABBS`, `CONTACT_POINTS`, . . .), one-shot `TestbedActionFlags` including `TAKE_SNAPSHOT` / `RESTORE_SNAPSHOT` (serialize the whole world — the germ of *reverse*) and `FRAME_SCENE` (auto-fit camera). Its `GraphicsManager` keeps hash maps from physics handles to render nodes so geometry is sent once and only poses update.

2.5 Open Source Physics — OSP / EJS (Java, Davidson College)

The classic *educational* library. `DrawingPanel` owns the pixel↔world transform with built-in pan/zoom; `PlottingPanel`/`Dataset` add live graphs; `DataTable` numeric readouts. `AbstractSimulation` runs an animation thread calling `doStep()` ∼10×/s, with a `SimulationControl` GUI of **Start/Stop,**

Step, Reset, Initialize buttons, an editable parameter table and a message area; `stepsPerDisplay` decouples physics rate from display rate. Numerics live behind an `ODE`/`ODESolver` interface (Euler, Verlet, RK4, RK45, ...). EJS generates Java *or* JavaScript from the same model.

2.6 SIMple Physics (Rust, educational)

High-school-oriented Rust simulators (SIMple Mechanics, SIMple Gravity) built on ggez + specs ECS + imgui + Lua scripting. Editable globals (`GRAVITY`, `PAUSED`, `DT`), per-object property panels (right-click), keyboard object creation, Space pause, **graph object properties and export to CSV** for lab reports, Lua scene presets. Its lesson: make every quantity inspectable and tweakable.

2.7 PhysicsHub (web, student-built)

Two related projects: a React/Next.js canvas app (`physicshub.github.io`) and the earlier Node/p5.js "The Physics Hub". The p5 version **layers canvases**: `bgCanvas` (chrome + menus), `simCanvas` (the simulation), `plotCanvas` (toggleable live plots), plus grid overlays; sliders/buttons via p5.gui; theory text with MathJax beside every simulation. The browser's `requestAnimationFrame` is the render loop; widgets mutate parameters read by the next frame.

2.8 The comparison chart

(Split across two tables to stay readable; same columns.)

	PyChrono	**PyBullet**	**SOFA**	**Rapier**
Language / stack	C++ (+SWIG Python)	C++ engine, Python API	C++ core, Python scenes	Rust
Rendering	Irrlicht / VSG / OpenGL	OpenGL debug renderer	Qt + libQGLViewer	Bevy/wgpu testbed, WASM demos
Process model	in-process	**client–server** (shm / UDP / TCP)	in-process	in-process (lib headless)
Scene / camera API	`ChVisualSystem*`, `AddCamera`	`resetDebugVisualizerCamera`, `configureDebugVisualizer`	Camera graph panel, Save/Reset View	`GraphicsManager` maps, `FRAME_SCENE`
Time stepping	`DoStepDynamics(dt)` user loop	`stepSimulation()`, `setTimeStep`	Animate / Step / dt field	`RunMode` {Running, Stop, Step}
Pause / step / reverse	gate the loop / —	call-when-you-want / —	buttons / —	RunMode / **snapshot+restore**
Mouse convention	L-rotate, R-pan, wheel	L-orbit, wheel, pan	L-rotate, R-pan, wheel	orbit / pan / zoom + pick
Toolbar / HUD	info panel (`i`)	sliders, preview tiles	Animate bar, FPS+time	egui panel, stats flags
Async sim↔GUI	none	**command→status mailbox**	none	ECS systems

	OSP / EJS	**SIMple Physics**	**PhysicsHub**
Language / stack	Java (EJS→JS too)	Rust (ggez, specs, Lua)	JS (React/Next or p5.js)
Rendering	Swing `DrawingPanel`	ggez 2-D	HTML `<canvas>`
Process model	in-process	in-process	in-browser
Scene / camera API	`DrawingPanel` pan/zoom, `InteractivePanel`	imgui panels, Lua globals	per-sim canvas + sliders
Time stepping	`doStep()` thread, `stepsPerDisplay`	`update()` each frame, `DT` global	`requestAnimationFrame`
Pause / step / reverse	Start-Stop-Step-Reset / —	Space pause / —	play-pause-reset / —
Mouse convention	drag interactives	L-drag move, R-click edit	widget-based
Toolbar / HUD	buttons + param table + messages	imgui property panels	sliders, plots, theory panes
Async sim↔GUI	animation thread	shared ECS state	same-thread JS

2.9 The "scene science" concept tree

Which simulator taught us each concept (→ = adopted in posim):

```
scene science
|-- scene lifecycle
|   |-- create / destroy a window           <- PyBullet connect()/disconnect()  ->
    SCENE CREATE / CLOSE
|   |-- describe geometry once, poses often <- Rapier GraphicsManager           -> init
     vs frame messages
|   |-- declarative scene description       <- SOFA .scn / Bullet URDF          -> init
     entity list (JSON)
|   `-- refresh / redraw on demand          <- SOFA Reset Scene                 ->
    SCENE REFRESH / REDRAW
|-- camera control
|   |-- orbit camera (yaw,pitch,dist,target)<- PyBullet resetDebugVisualizerCamera ->
    SCENE ROTATE / server camera
|   |-- translate / pan                     <- Irrlicht & QGLViewer right-drag  ->
    SCENE TRANSLATE, arrows, shift-drag
|   |-- zoom (wheel + keys)                 <- all seven                        ->
    SCENE ZOOM IN/OUT/<f>, wheel, +/-
|   `-- frame-the-scene auto-fit            <- Rapier FRAME_SCENE               ->
    fit_distance() at create/refresh
|-- playback control
|   |-- run-mode state machine              <- Rapier RunMode                   ->
    Stopped/Running/Paused/Reversing
|   |-- start / stop / pause / single-step  <- OSP AnimationControl             ->
    SCENE START/STOP/PAUSE, steps
|   |-- reverse via snapshot history        <- Rapier TAKE/RESTORE_SNAPSHOT     ->
    SCENE REVERSE (ring buffer)
|   `-- settable time step                  <- PyBullet setTimeStep / SOFA dt   ->
    SCENE SET_TIME_STEP
|-- asynchronous messaging
|   |-- command -> status protocol          <- PyBullet client-server           ->
    WebSocket JSON both ways
|   |-- window -> notebook events           <- PyBullet debug/user events       ->
```

```
    SCENE EVENTS + {"event":...}
|   `-- transport-agnostic message structs  <- PyBullet shm/UDP/TCP             -> same
     JSON, any front end
`-- educational UX
    |-- toolbar of big obvious buttons      <- OSP SimulationControl            -> the
    toolbar
    |-- status readouts (t, E, fps)         <- SOFA FPS/time, OSP DataTable     -> the
    status bar
    |-- trails / plots / layered canvas     <- PhysicsHub plotCanvas            -> grid
     + trails + bodies layers
    |-- hide/show and visual flags          <- Bullet configureDebugVisualizer  ->
    SCENE HIDE / SHOW
    `-- inspect-everything philosophy       <- SIMple Physics                   ->
    labels, help overlay, STATUS
```

3 Porting recommendations (and what posim actually did)

Constraint recap: posim allows **zero external crates, zero `unsafe`, zero warnings**. That immediately rules some things in and some out.

3.1 Recommended and adopted

idea	source	posim realization
Command→status message protocol	PyBullet	Every window action is a small JSON command; the simulator answers with state. Maps 1:1 onto WebSocket text frames — `std::net::TcpListener` + a hand-rolled RFC 6455 layer (SHA-1 + base64 included, ∼200 lines, fully unit-tested).
Engine/renderer decoupling	Rapier	The physics core never draws. The scene server owns a *synchronized copy* of the system; the browser is just a view.
Geometry once, poses per frame	Rapier `GraphicsManager`	`init` message carries shapes/sizes once; 30 Hz `frame` messages carry only `[x,y,z,qw,qx,qy,qz]` per body.
Run-mode state machine	Rapier `RunMode`	`Stopped / Running / Paused / Reversing` enum drives the playback thread.
Reverse via snapshots	Rapier snapshot / restore	A bounded ring buffer (20,000 frames) of cloned system states; `SCENE REVERSE` replays it backward and auto-pauses at the beginning. Cheap because `PhysicalObjectSystem` is `Clone`.
Control-panel UX	OSP `SimulationControl`	Toolbar: Start/Pause/Stop/Reverse/step/dt — the exact OSP button set plus reverse.
Status readouts	SOFA, OSP	Status bar: mode, t, dt, E, bodies, hidden, history, camera, fps.
Camera conventions	Irrlicht / QGLViewer / Bullet	Left-drag orbit, wheel zoom, right/shift-drag pan, arrows translate — the cross-simulator standard.
Auto-fit camera	Rapier `FRAME_SCENE`	`fit_distance()` = max(12, 2.5× farthest entity).
Layered drawing	PhysicsHub	Grid layer → trails layer → bodies (painter-sorted) → labels.
Settable dt	PyBullet `setTimeStep`	`SCENE SET_TIME_STEP <dt>` and the toolbar dt field (validated: positive, finite).
Debug/user events	PyBullet	Window errors and actions become notebook events (§7).

3.2 Considered and deliberately NOT ported

idea	why not
Shared-memory transport (PyBullet)	Requires raw memory mapping — impossible without `unsafe` or an external crate. TCP on localhost is fast enough for 30 Hz frames.
Native GUI toolkits (Qt, Irrlicht, Bevy, egui, ggez)	All are external dependencies. The browser is the one universally-available renderer; the page is embedded in the binary with `include_str!`.
WebGL/GPU rendering	Needs no dependency, but canvas-2D with manual projection is simpler, debuggable, and fast enough for classroom scenes. A WASM/WebGL path remains open (Rapier's route).
Lua scripting (SIMple)	posim already has its own command language; two languages would confuse the audience.
Fragmented/binary WebSocket frames	Rejected explicitly by the server: text-only keeps the protocol inspectable by students.

4 The SCENE command family (complete reference)

All keywords are case-insensitive. Numeric arguments are *term-level* expressions: `-5` is negative five, `2*2` works; write `(1+2)` to use a sum. Full grammar in `grammar.pdf` §3.

command	what it does
`SCENE CREATE [port]`	Start the scene server (OS-assigned port if omitted), open the browser page, show all entities. Second call reports the existing URL.
`SCENE CLOSE` (alias `SCENE DESTROY`)	Shut the server down and disconnect every window.
`SCENE TRANSLATE dx dy [dz]`	Move the camera target by a world-space offset (`dz` defaults to 0).
`SCENE ROTATE dyaw dpitch`	Orbit the camera: azimuth and elevation, in degrees (pitch clamps to $\pm 89°$).
`SCENE ZOOM IN` / `SCENE ZOOM OUT`	Zoom by a fixed factor (1.25×).
`SCENE ZOOM <f>`	Zoom by factor `f` (> 1 zooms in; e.g. `scene zoom 0.5` zooms out).
`SCENE HIDE [n|ALL]`	Hide object `objN`, or everything (default ALL).
`SCENE SHOW [n|ALL]`	Show object `objN`, or everything (default ALL).
`SCENE REFRESH`	Re-copy the notebook's current system into the window (clears history).
`SCENE REDRAW`	Re-send the full scene description to every connected window.
`SCENE START`	Begin (or resume) forward time-stepped evolution.
`SCENE STOP`	Halt evolution and clear the recorded history.
`SCENE PAUSE`	Freeze evolution; `SCENE START` resumes, history kept.
`SCENE REVERSE`	Play **backward in time** through the recorded history; pauses automatically at the beginning. Errors if no history exists yet.
`SCENE RESET`	Re-initialize the playback: **every mutable value and the time return to their initial values** — the state last synced at `SCENE CREATE`/`SCENE REFRESH` — bit-identically; history and the step counter clear and the mode returns to Stopped. `SCENE START` (or the window's Start button) then re-runs the simulation from the beginning. The toolbar **Reset** button calls the very same primitive.
`SCENE SET_TIME_STEP dt`	Set the playback time step (must be positive and finite).
`SCENE STATUS`	Report URL, connected windows, mode, t, dt, steps, history size, entities, hidden list, camera.
`SCENE EVENTS`	Drain and print the asynchronous window → notebook event queue.

Notes for the curious:

- The playback loop runs on its own thread at ∼30 ticks/s (33 ms) and advances a **synchronized copy** of your system — your notebook state is never mutated by the window. `SCENE REFRESH` re-syncs the copy; a future `RUN`/`STEP` in the notebook does not disturb a running scene.
- Every forward step goes through `physical_object::integrate` — the same sundials (CVODE/ARKODE) code path as `STEP`/`RUN`. There is no second, lesser integrator hiding in the graphics.
- `RESET` in the notebook keeps the window open and re-syncs it to the now-empty system — including clearing the box wireframe and wall flags if a `BOX` existed.
- `SCENE RESET` is **not** the notebook's `RESET`: the notebook command empties the system itself, while `SCENE RESET` only rewinds the window's playback copy to the state it was last synced with. The reply spells it out: `scene playback reset to its initial state (t = 0, 1 entity); Start runs the simulation again from the beginning`.

5 The wire protocol (WebSocket, JSON text frames)

The page connects to `ws://127.0.0.1:<port>/ws`. One JSON document per frame, both directions.

5.1 Simulator → window

```
{"type":"init","t":0.0,"dt":0.01,"mode":"stopped",
 "camera":{"yaw":-60.0,"pitch":55.0,"dist":12.0,"target":[0,0,0]},
 "hidden":[],
 "entities":[{"i":0,"mass":256.0,"charge":0.0,"shape":"sphere","radius":0.35},
             {"i":3,"mass":2.0,"charge":0.0,"shape":"cuboid",
              "half_extents":[0.3,0.2,0.1]}]}
```

Sent on connect, on `SCENE REFRESH`/`REDRAW`, after hide/show, and after a playback reset (`SCENE RESET` / the Reset button). Geometry travels **once** (the Rapier idea). Note that `BOX <size>` / `BOX OFF` in the notebook do **not** push a new init on their own — box changes reach an open window only via `SCENE REFRESH`, and the `BOX` reply reminds you: `(scene window open: SCENE REFRESH shows the box)`. The one exception is `RESET`, which re-syncs the window itself and clears the box wireframe and wall flags automatically.

Each entity carries its shape-specific geometry: a `sphere` its `radius`, a `cuboid` its `half_extents`, a `torus` its `ring_radius` and `tube_radius`, a `disk` its `radius`, a `cylinder` its `radius` and `half_height`, and — new in the dumbbell release — a `dumbbell` its two sphere radii `r1`/`r2`, its `rod_radius`, and the sphere-centre offsets `z1`/`z2` along the local axis (the local origin is the composite COM, so the offsets are mass-weighted and unequal). Any entity whose object was registered with `NEW ... AS <name>` also carries `"name"` — the window prefers it over `objN` for the label:

```
{"i":6,"mass":1.0,"charge":0.0,"shape":"torus","ring_radius":1.5,"tube_radius":0.5}
{"i":9,"mass":0.6666666666666666,"charge":0.0,"shape":"disk","radius":1.0}
{"i":11,"mass":2.0,"charge":0.0,"shape":"cylinder","radius":0.5,"half_height":0.75}
```

```
{"i":0,"mass":0.0,"charge":0.0,"shape":"cuboid","half_extents":[1.0,4.0,4.0],"wall":
    true}
{"i":0,"mass":3.5,"charge":0.0,"shape":"dumbbell","r1":0.25,"r2":0.25,
 "rod_radius":0.1,"z1":-0.6428571428571429,"z2":0.35714285714285715,
 "name":"dumbell0"}
```

Two optional fields describe the rigid bounding box: each of the six static wall slabs carries `"wall":true` on its entity (the page then skips it in the body-drawing pass), and the message top level gains `"box":4.0` — the inner side length — **absent** when no box exists.

```
{"type":"frame","t":1.234,"dt":0.002,"mode":"running","steps":617,
 "history":617,"energy":-592.49,
 "p":[0.15,0.0,0.0],"l":[0.4443,0.0,-1.506],"hidden":[],
 "contacts":[{"i":0,"j":1,"point":[0,0,0],"normal":[1,0,0],"impulse":2.0}],
 "bodies":[[x,y,z,qw,qx,qy,qz], ...]}
```

Broadcast every tick (∼30 Hz) while any window is connected. Bodies are ordered by entity index; quaternions are **w-first** (the posim convention everywhere). `contacts` carries every collision the playback copy resolved in its latest step — `normal` is the unit action–reaction line from body `i` toward body `j` — and the page draws each one as a fading golden arrow through the contact point (arrowhead on the `j` side, ∼0.6 s persistence; toggle with **Contacts** / `C`). Verified live: a head-on elastic impact broadcasts `normal [1,0,0]`, `point [0,0,0]`, `impulse 2.0` — the exact analytic values — and the arrow renders at the projected contact point.

Alongside `energy`, every frame now carries the other two conserved totals, computed on the playback copy every tick: `p` is the **total linear momentum** and `l` the **total angular momentum about the origin** (each `[x,y,z]`). These are exactly the numbers the "conserved quantities" readout (§1.2) displays.

```
{"type":"camera","camera":{"yaw":...,"pitch":...,"dist":...,"target":[...]}}
```

Sent when the *notebook* moves the camera (`SCENE TRANSLATE/ROTATE/ZOOM`) so every window follows.

5.2 Window → simulator

```
{"type":"cmd","action":"start"}                  // toolbar buttons:
   // start | pause | stop | reverse | reset | step | step_back | set_dt | refresh
{"type":"cmd","action":"set_dt","value":0.005}
{"type":"camera","yaw":-92,"pitch":71,"dist":7.9,"target":[0,0,0]}
   // gesture sync, throttled to 10 Hz, so SCENE STATUS matches the screen
{"type":"event","level":"error","message":"..."}  // e.g. page JS errors
{"type":"request_state"}                          // ask for a fresh init
```

Malformed messages never crash the server; they become error events the notebook can read.

6 Under the hood (for the reader who wants the machinery)

```
posim process
```

```
|-- VM thread (your commands)           SCENE ... --+
|-- listener thread (TcpListener, non-blocking poll) | Arc<Mutex<Shared>>
|-- playback thread (33 ms tick)  <------------------+
|     Running   -> history.push(system.clone()); integrate::step(dt)
|     Reversing -> system = history.pop_back()   (auto-pause at start)
|     broadcast frame JSON to every outbox (mpsc channels)
`-- per-window threads
      reader: parse WS frames -> handle_client_message (0.5 s poll for shutdown)
      writer: outbox receiver -> WS text frames (write half mutex-shared
              with the reader so pings/close never interleave a frame)
```

- **WebSocket layer** (`posim/src/scene/ws.rs`): SHA-1 (FIPS 180-4) and base64 (RFC 4648) implemented from the standard, plus RFC 6455 framing (text/close/ping/pong; masked client frames; 126/127 length forms). Unit-tested against the official test vectors, including the RFC's own handshake example (`dGhlIHNhbXBsZSBub25jZQ==` → `s3pPLMBiTxaQ9kYGzzhZRbK+xOo=`).
- **HTTP layer**: `GET /` serves the embedded page (`include_str!`), `GET /ws` upgrades. Anything else: 404. Bound to `127.0.0.1` only — the window is local by construction.
- **The page** (`posim/src/scene/scene.html`): one self-contained HTML/CSS/JS file. Canvas-2D renderer with a z-up orbit camera, perspective projection, painter-sorted bodies, radial-gradient spheres, quaternion-rotated shape wireframes, adaptive grid, world axes, 800-point trails. No frameworks, no network fetches.
- **Shape wireframes** — every extended shape is drawn as a small set of loops and lines, all rotated by the body quaternion so spin is visible: a cuboid keeps its 12-edge wireframe; a **torus** is its outer and inner equators, two tube rings (the tube's top and bottom circles) and four tube cross-sections — enough to read the hole and the spin; a **disk** is its rim plus two diameters; a **cylinder** is its two cap rims plus four side lines.
- **The dumbbell renderer** — a **dumbbell** is drawn as one rigid body: two radial-gradient-shaded spheres (radii `r1`, `r2`) at their COM offsets `z1`, `z2` rotated by the body quaternion, joined by the rod's **four silhouette lines** (the lines between the sphere centres, offset radially by `rod_radius` in four directions). Because the offsets are mass-weighted the heavier end sits closer to the drawn position — the spin and the asymmetry are both visible.
- **The conserved-quantities readout** — a permanent labeled overlay (`id="hud"`, top-left under the toolbar) rebuilt on every frame from the frame message's `energy`, `p` and `l` fields: an `E` line (8 digits) and `P`/`L` lines showing the three components plus the `|.|` magnitude (5 digits). Entity labels, drawn in the body pass, prefer the init entity's `"name"` (the user's `NEW ... AS` registration) and fall back to `objN`.
- **`Shared.initial` + `reset_playback`** — the playback state keeps an `initial` snapshot: the system as last synced from the notebook (set at `SCENE CREATE` and by every `SCENE REFRESH`/`RESET` re-sync). `reset_playback` restores it as a bit-identical clone — every mutable value and the time return to their initial values — clears the history ring and the step counter, returns the mode to Stopped, and flags a fresh init for every window. Three doors, one primitive: the toolbar **Reset** button, the window's `reset` command, and the notebook's `SCENE RESET` all land in the same function.
- **The rigid bounding box** — when the init message carries `"box"`, the page strokes the interior cube as a dashed `#5d84a8` wireframe (12 edges, drawn just after the grid, beneath trails

and bodies). The six static wall slabs that implement the box arrive tagged `"wall":true` and are skipped in the body-drawing pass: the dashed wireframe represents them.

7 Asynchronous data in both directions

The old design was strictly synchronous (one reply per request). The scene link is genuinely asynchronous:

Notebook → window. Scene commands broadcast immediately; a running playback streams frames whether or not the notebook is busy.

Window → notebook. The window pushes errors, data requests and user actions at any time. They are queued (bounded at 1,000) and reach you two ways:

1. **Interactive / script mode** — type `SCENE EVENTS`:

```
In [7]: scene events
Out[7]: window connected (1 total)
        window action: start
        error: canvas exploded            <- a page JS error, reported home
```

2. **`-machine` / Jupyter mode** — posim emits *unsolicited* lines `{"event":"scene","message":"..."}` between replies. The Jupyter wrapper kernel runs a background reader thread that routes real replies to the requester and pushes event lines straight to the notebook front end as `[scene] ...` stream output — errors arrive on stderr (red), everything else on stdout. The `{"op":"events"}` request also drains the queue explicitly.

8 How it was verified

Rust tests (52 total, `cargo test -workspace`). New for the scene subsystem: SHA-1/base64/accept-key official vectors; an HTTP test that the served page contains the toolbar, status bar and all three gesture handlers; a full WebSocket session test (handshake → init → camera sync → start/pause → event queue → frame broadcast); a playback test proving forward-then-reverse lands back *exactly* on the start state; lexer/parser/VM tests for every `SCENE` command, including the errors (`scene rotate 15` missing an argument, port > 65535, commands before `SCENE CREATE`).

Wire-protocol test (`jupyter/test_protocol.py`, stdlib only): 24 checks green, including the whole `SCENE` family end-to-end over `-machine` and the `events` op.

Kernel test (`jupyter/test_kernel.py`, real ZMQ): 5 checks green.

Real-browser verification (headless Chrome 150 driven over the DevTools protocol, dispatching genuine OS-level input events): 16/16 checks passed —

```
ok  scene shows all simulator entities (4)
ok  statusbar reports connection          ok  toolbar is present
ok  ArrowRight translates the view        ok  ArrowLeft translates it back
ok  ArrowUp/ArrowDown also translate
ok  left-drag rotates (yaw -92 -> -124)   ok  left-drag rotates (pitch 71 -> 87)
ok  drag does not zoom
ok  wheel up zooms in (12.00 -> 7.89)     ok  wheel down zooms out (7.89 -> 12.00)
ok  keyboard + zooms in                   ok  keyboard - zooms out
```

```
ok  toolbar Start begins evolution        ok  toolbar Pause freezes it
ok  toolbar Reverse plays backward (t 0.084 -> 0.022)
```

The gestures were then **independently re-verified** in a second, different browser session (2026-07-22, driving genuine key, drag and wheel events against a live 4-entity scene on port 7878): ArrowRight moved the camera target along the view's right axis and ArrowLeft returned it **exactly** to `[0, 0, 0]`; a left-drag changed yaw $-60° \to -98.8°$ and pitch $55° \to 32.2°$ while distance and target stayed untouched (a pure orbit); wheel-up/down zoomed by the exact 1.15 factors ($12 \to 10.435 \to 12$) and the `+/-` keys by the exact 1.2 factors ($12 \to 10 \to 12$); toolbar Start advanced t through the CVODE playback path, Pause froze it, and Reverse replayed history backward until it auto-paused at **exactly** $t = 0$ with the initial energy $E = -194.84952$ restored to the last digit.

The four self-checking physics examples (Kepler, outer solar system, tumbling body, charged particle) still print SUCCESS, the build is warning-free, and `Cargo.lock` still lists exactly the five local crates — the zero-dependency proof.

Shapes-and-box release (2026-07-24). Two further layers of checks accompany the `TORUS`/`DISK`/`CYLINDER` shapes and the `BOX` command (the workspace suite is now 94 tests: 37 lib + 15 collision + 9 conservation + 33 posim, zero warnings):

- **The init message carries the new geometry.** The new posim test `box_shapes_and_wall_flags_reach_` opens a genuine WebSocket to the scene server and asserts the init JSON contains `"box":4.0`, `"shape":"torus"` with `"ring_radius":1.5` and `"tube_radius":0.5`, and `"wall":true` on the static slab.
- **Live browser session.** The 12-entity demo (`scripts/collisions/11_box_of_shapes.posim`: six walls plus a torus, point, sphere, disk, cube and cylinder inside `BOX 4`) was played back in a real browser window: the window's playback copy conserved energy to ∼`1e-9`; a pixel scan of the canvas found the dashed box wireframe (5,904 pixels of `#5d84a8`) and the golden contact arrows (2,218 pixels); toolbar Start and Reverse were exercised through real DOM clicks; the JS console stayed free of errors.

Reset + conserved-quantities release (2026-07-24). The workspace suite is now **103 tests: 40 lib + 16 collision + 9 conservation + 38 posim**, zero warnings. What the new layers pin down:

- **Reset is a true re-initialization.** The posim test `reset_restores_the_initial_state_and_start_reru` runs the playback forward (≥ 10 steps), calls the reset primitive, and asserts the mode is Stopped, the time is back to its initial value, the packed 13-per-object state is **bit-identical** to the initial pack, and the step counter is 0 — then sets Running again and asserts the simulation re-starts. The page-serve test additionally asserts the permanent `id="bt-reset"` button is in the served HTML, and `scene_commands_compile` covers `SCENE RESET`.
- **Reset verified live.** In a real browser session the demo was run to $t = 0.72$, the toolbar Reset was clicked: mode `stopped`, $t = 0$, history = 0, and the body's state bit-identically back at its start values; clicking Start then set the window running again from the beginning.
- **The protocol carries the new fields.** The WebSocket session test (`websocket_session_end_to_end`) asserts every broadcast frame contains `"p":[` and `"l":[`; `box_shapes_and_wall_flags_reach_the_init_me` asserts the init JSON carries the registered user name (`"name":"ringo"`).

- **The dumbbell demo, live.** The two-dumbbell collision demo (`scripts/collisions/12_two_dumbbells.p` — two user-named dumbbells built by a user-defined function, colliding off-center with spin) was played back in a real browser window: the entity labels read `dumbell0` and `dumbell1`, and the conserved-quantities readout showed

```
E = 7.86531061, P = [0.15000, 0.00000, 0.00000] |.| = 0.15000, L = [0.44430,
    0.00000, -1.50600] |.| = 1.57017
```

  identically before and after the impact (L's y component showed `-1.31228e-13` after — machine-epsilon noise). The notebook run of the same script reports, through 2 real CVODE collision events,

```
t = 3 (3861 solver steps, 60 snapshots, |dE/E| = 9.463e-11, 2 collision(s) --
    CONTACTS lists them)
```

  with the total momentum `[0.15000000000000036, 0, 0]` **bit-identical** before and after.

9 FAQ / troubleshooting

No window appeared after `SCENE CREATE`.
The command prints the URL (e.g. `http://127.0.0.1:41234/`). Open it in any browser yourself — the automatic opening uses `xdg-open`, which headless machines lack.

I want a predictable port.
`SCENE CREATE 7878` → the URL is always `http://127.0.0.1:7878/`. If the port is busy you get a clear error; pick another or use `0` (OS-assigned).

Stop the browser from opening (tests, servers).
Set the environment variable `POSIM_NO_BROWSER=1`.

`REVERSE` says there is nothing to reverse.
Reverse replays *recorded* history. Run `SCENE START` (or single steps) first; `STOP` clears the recording, `PAUSE` keeps it.

The window shows old objects.
The window views a synchronized *copy*. After creating/deleting objects in the notebook, type `SCENE REFRESH`.

Is any of this sent over the internet?
No. The server binds `127.0.0.1` (loopback) only.

Where do error messages from the window go?
Into the event queue: `SCENE EVENTS` in the notebook, `[scene] ...` lines in Jupyter, `{"event":...}` lines in `-machine` mode.